資料庫設計：資料連動 (縣市、汽車鑑價網)

資料表總覽
taxonomies: 品牌、車型、車款、排氣量、年式、出廠年份
terms: Toyota, Honda, Nissan, Audi, BMW, Benz, Tesla, Ford..., 一般房車、suv, 掀背車..., 1800cc, 2000cc...
cars
car_items
car_links
五張表搞定全部。

資料結構設計

- **`taxonomies` 表：定義分類類型**

id	name	slug	description
1	品牌	brand	-
2	車型	category	-
3	車款	model	-
4	排氣量	engine	-

`terms` 表：定義分類項目（詞彙）

id	taxonomy_id	name	slug	sort_order
1	1	Toyota	toyota	1
2	1	Honda	honda	2
3	2	4門	4-door	1
4	2	SUV	suv	2
5	3	Altis	altis	1
6	3	CR-V	crv	2
7	4	1.8L	1.8	1
8	4	2.0L	2.0	2

cars

`cars` 表：車輛主表

id	title	price	year	...
1	Toyota Altis 2020	500000	2020	...

cars_terms

- `car_term` 中介表：車輛與詞彙的多對多關聯

id	car_id	term_id
1	1	1 (Toyota)
2	1	3 (4門)
3	1	5 (Altis)
4	1	8 (2.0L)

以上不搭配 car_links 資料表的情況下，是【資料型驅動】，從現有的二手車裡面，找出存在的值。

每一個 term 的詞義都是全局唯一，例如整個 terms 只會有一筆4門，而不是 toyota 有自己的4門，nissan 也有自己的4門。

✅ 想查出所有品牌： php 複製 編輯

```
$brands = Term::where('taxonomy_id', Taxonomy::where('slug', 'brand')-
>first()->id)->get();
```

✅ 使用者選擇 Toyota → 查出可選的「車型（4門/5門）」： php 複製 編輯

```
// 找出 toyota 這個 term
$termToyota = Term::where('slug', 'toyota')->first();

// 根據 toyota, 找出此品牌全部的車
$carIds = DB::table('car_term')->where('term_id', $termToyota->id)-
>pluck('car_id');

// 再次查詢 car_term，使用所有 toyota 的 car_id，並且 taxonomy_id=2(車型)
$categoryTermIds = DB::table('car_term')
->whereIn('car_id', $carIds)
->whereIn('term_id', Term::where('taxonomy_id', 2)->pluck('id'))
->distinct()
->pluck('term_id');

// 如此，得到所有 Toyota 的車型
$categories = Term::whereIn('id', $categoryTermIds)->get();
```

以上得到的資料結果，跟鑑價網一樣，會是存在的二手車。如果某個項目底下沒有二手車，就不會有選項。
例如 Toyota 的車型有 5 門(對車不熟，先假設有 5 門)，但是 5 門底下目前沒有任何二手車要出售。所以 Toyota 的車型選單，就不會有 5 門這個項目。

【結構型驅動】

外優化（選越多、結果越準）

每選一個條件（例如品牌、車型、車款），你的查詢都可以：

```
$selectedTermIds = [1, 3, 5]; // ex: Toyota + 4門 + Altis
```

```
$carIds = DB::table('car_term')
->whereIn('term_id', $selectedTermIds)
->groupBy('car_id')
->havingRaw('COUNT(DISTINCT term_id) = ?', [count($selectedTermIds)])
->pluck('car_id');
```

這樣你就拿到「同時符合這些條件」的車輛 ID，接下來再查其他 term。

解法一：切換成「結構驅動型」

不是根據「現有車輛資料」，而是根據「你系統內建的 term 結構」 → 不管有沒有車，都先顯示所有 term 給使用者選。

實作方法：

當使用者選 `品牌 = Audi` ，你可以照樣查出：

```
// Audi 的 term_id
$audiTermId = 12;

// 查所有 term_id = 12 的品牌所對應的「車型」有哪些
// ❌ 如果用車資料反查就會查不到
// ✅ 所以改查一張「品牌 → 車型關聯表」或事先定義的 term_term 關係
```

加一張 `term_links` 中介表，來描述關聯：

id	parent_term_id	child_term_id	taxonomy_id
1	12 (Audi)	21 (4門)	車型
2	12 (Audi)	22 (SUV)	車型
3	12 (Audi)	35 (A6)	車款

這樣你就可以做到：

- 即使沒有實際車輛資料
- 也能根據 term 結構顯示連動下拉選項

🧠 解法二：兩種混合

你也可以選擇**前端顯示所有 term，但加上可用性標記**：

- 先查出所有「品牌為 Audi 的車型 term」

- 如果查不到車，前端還是顯示，但附註「目前無資料」或 disable

資料型驅動有一個缺點：如果 toyota 的車很多，這樣過程中會找出所有 car_id, 再去做 whereIn(), 比較耗費記憶體？

非常精闢的觀察！💡
是的，你的這段程式碼在邏輯上沒錯，但的確有個 **潛在效能問題**：

🧠 問題點：`pluck('car_id')` → `whereIn('car_id', [...])` 可能導致記憶體爆炸

當：

- Toyota 的車數量很多（例如上千輛以上）
- `pluck('car_id')` 把所有 ID 都取出來存在 PHP memory 裡
- 再 `whereIn(car_id, [...])` 時把整個 array 傳給 SQL

這會導致：

- PHP 端記憶體壓力大
- SQL 查詢效率低（`WHERE IN (...)` 過長可能拖慢 query planner）

✅ 更有效率的寫法：使用 子查詢（Subquery）

Laravel 支援這種寫法，讓你避免將資料撈回來再傳進去：

```
php
複製編輯
$termToyota = Term::where('slug', 'toyota')->first();

$toyotaCarIdsSubquery = DB::table('car_term')
->select('car_id')
->where('term_id', $termToyota->id);

$carModelTermIds = DB::table('car_term')
->whereIn('term_id', Term::where('taxonomy_id', 2)->pluck('id'))
->whereIn('car_id', $toyotaCarIdsSubquery) // 🔥 用 subquery
->distinct()
->pluck('term_id');

$categories = Term::whereIn('id', $carModelTermIds)->get();
```

這樣就讓整個查詢邏輯 **都在 DB 裡完成，不落地到 PHP memory**，效能更穩定、可擴充性更高。

🚀 更進一步：用 JOIN 版本（效能更極致）

如果你希望進一步讓 DB 引擎自己做最佳化，可以寫 JOIN 查詢（支援 Laravel Query Builder）：

```
php
複製編輯
$termToyota = Term::where('slug', 'toyota')->first();

$categoryTermIds = DB::table('car_term as ct1')
->join('car_term as ct2', 'ct1.car_id', '=', 'ct2.car_id')
->where('ct1.term_id', $termToyota->id)
->whereIn('ct2.term_id', Term::where('taxonomy_id', 2)->pluck('id'))
->distinct()
->pluck('ct2.term_id');

$categories = Term::whereIn('id', $categoryTermIds)->get();
```

🔚 總結

寫法	效能	是否佔用 PHP 記憶體	可讀性	備註
`pluck → whereIn`	❌ 中等偏低	✅ 高	✅ 易懂	初學常用，但不建議用在大量資料
`subquery`	✅ 中等偏高	❌ 低	✅ 中	最推薦的實務寫法
`JOIN`	✅✅ 高	❌ 低	⚠️ 較複雜	需要小心 index，但效能最強

如你想用 Eloquent 版本、加 cache、或做 eager load，我也可以幫你擴充 👍